Objectilang en Rascal – Entrega 3: tipos con TypePal

ISIS-2111 Elementos Esenciales de Lenguajes de Programación – Universidad de los Andes

Integrantes:

Iker Sebastián Barbosa Cespedes (202424135)

Joan Ortiz Celis (202515912)

Cambios respecto a la Entrega 2

La línea base es la gramática del documento de la Entrega 2. Agregado (A), Eliminado (E) o Modificado (M):

	Cambio	Detalle
M	for	ClausulaFor ::= Patron "<-" Exp | IDENTIFICADOR "in" Exp. Antes solo aceptaba Patron "in" Exp.
E	Regla Llamada	Llamada ::= "(" Argumentos? ")" | "." IDENTIFICADOR ya no se usa. La reemplazan las alternativas de Exp de la fila siguiente.
A	variable, llamada, acceso en Exp	Una variable (x) y una llamada (f(x)) son nodos distintos. El acceso a miembros es Exp "." IDENTIFICADOR ArgsParen? y lleva el receptor, en lugar de una lista plana de sufijos.
A	Cuerpo de objeto	DefinicionObjeto acepta CuerpoDeObjeto?, con MiembroObjeto ::= DeclaracionVariable | DefinicionMetodo.
A	Operadores in, ->, =>	En el nivel de comparaciones, junto a <, >, =, etc.

	Cambio	Detalle
M	neg	Negacion es un léxico "neg" !>> [a-zA-Z0-9]. Así negate es un nombre válido.
M	Todas las palabras clave	Se les agregó !>> [a-zA-Z0-9] (new, val, var, and, or, in, if, then, else, for, object, extends, method, override, main). Antes newton, newPerro y varianza = 5 eran ambiguos, y a andrea se leía como a and rea.
M	Interpolación	"$" "{" Layout Exp Layout "}": acepta espacios, como ${ a + 1 }.
A	Asignación	Sentencia ::= ... | IDENTIFICADOR "=" Exp. Ver sección 3.
M	AST: cadenas y caracteres	"hola" queda dCadena("hola") y 'x' queda dCaracter("x"), sin comillas (ASTBuilder.quitarComillas).
A	AST: nodos nuevos	variable, acceso, sentAsig, pertenece, flecha, flechaDoble, y el cuerpo de objeto.
M	Main.rsc	Ya no es la plantilla: main() lee un archivo .ol, verifica los tipos (Checker.verificar) y, si no hay errores, lo ejecuta y muestra el resultado en consola. mostrarAST imprime el AST.
M	if-then-else (expSi)	Pasa al nivel de menor precedencia de Exp: if c then a else b + d se lee como else (b + d), y no como (if c then a else b) + d.

	Cambio	Detalle
M	Parser.rsc	Se agregan parsearTexto, parsearOverride y parsearCuerpo, que parsean texto en memoria con la misma política de ambigüedades que un archivo.
A	CodeGenerator, JavaGenerator, Interpreter	Punto 2: del AST a texto Objectilang, a Java y ejecución directa. Ver la sección Generación de código.

Problemas de la gramática que se documentan

El separador ";" entre sentencias sigue siendo obligatorio

Los snippets oficiales separan sentencias con saltos de línea, y esta gramática exige ;. Se intentó hacerlo opcional con (Sentencia ";"?)* y con un separador que puede ser vacío, y ambas versiones dan ambigüedad en todos los snippets, incluso en snippet2, que tiene una sola sentencia. Causas: print("hola") se lee como una llamada, o como la variable print seguida de la sentencia ("hola"). obj.metodo(x) y new T(x) tienen el mismo problema, porque los argumentos son opcionales.
a en una línea y -b en la siguiente se leen como a - b o como dos sentencias.
Con variable: Identificador !>> "(" pasan snippet2 y snippet4. Pero snippet1 y snippet3 siguen ambiguos: haría falta dividir en dos producciones el acceso y el new (con y sin argumentos), y la resta de línea nueva no se resuelve sin que el salto de línea sea significativo. Se decidió mantener ; obligatorio, que es lo que ya documentaba la Entrega 2. Con ; los cuatro snippets oficiales parsean.

Asignación frente a igualdad

El enunciado no define reasignación de variables; = aparece solo como comparación y en declaraciones. La retroalimentación de la Entrega 2 señaló que faltaba una sentencia de asignación, por lo que se agregó, como extensión mínima, sentAsig: IDENTIFICADOR "=" Exp. Como x = 2 es ambiguo entre asignación e igualdad, Parser.parsearArchivo elige la asignación a nivel de sentencia; dentro de una expresión sigue siendo igualdad (val b = x = 2). Cualquier otra ambigüedad sigue siendo error.

sentAsig acepta únicamente un identificador a la izquierda (x = expresión). Una línea como k.valor = 5; no se interpreta como asignación a un campo: se parsea como una comparación (k.valor = 5), por lo que no produce ambigüedad ni error y no modifica el campo. Tampoco existe asignación encadenada: x = y = 3 se lee como x = (y = 3), es decir, asignar un booleano a x, y el verificador de tipos lo reporta como error cuando x no es booleana.

Cambios a la gramática

Las anotaciones de tipo son opcionales en las declaraciones, así que los programas sin tipos siguen siendo válidos.

AnotTipo = ":" Tipo, opcional en val/var, en method, en Obj.m = method y en override. Ejemplo: val x: Int = 3.

Tipo es simple (Int, Float, Boolean, Char, String o el nombre de un objeto) o genérico (Lista[Perro], Transformador[Animal, ComidaPerro]).

ParametrosTipo declara los parámetros de tipo de un objeto con varianza: +A covariante, -A contravariante, A invariante.

Los parámetros de métodos y los campos del constructor llevan tipo obligatorio: (x: Int), (val elemento: A).

Un objeto puede tener cuerpo (CuerpoDeObjeto) con campos y métodos.

Definición del parser

Para esta entrega se implementó el parser de Objectilang en Rascal, reutilizando y arreglando la gramática desarrollada en la Entrega 2. El objetivo fue permitir que los programas escritos en archivos.ol pudieran ser reconocidos y transformados en un AST, utilizando los modulos ya implementados en la entrega pasada.

Durante la implementación se identificaron algunos problemas:

Separación de sentencias: se intentó permitir que el punto y coma fuera opcional, pero esto generaba ambigüedades entre llamadas a métodos, expresiones y sentencias. Por esta razón, se mantuvo el punto y coma obligatorio entre sentencias.

Asignación e igualdad: se agregó una producción para permitir la reasignación de variables mediante IDENTIFICADOR "=" Exp. Como esta construcción puede confundirse con una expresión de igualdad, se distingue entre la asignación a nivel de sentencia y la igualdad dentro de una expresión.

Generación de código

Para permitir la ejecución de los programas y mostrar los resultados en consola, tal como lo solicita el enunciado, se implementaron tres módulos que parten del AST que produce construirAST: CodeGenerator.rsc reconstruye el programa en texto Objectilang (generarDesdeArchivo). El texto generado se puede volver a parsear con parsearTexto; por eso el cuerpo de un método se genera con ; entre sentencias y sin ; después de la última, que es lo que admite la gramática.
JavaGenerator.rsc traduce el cuerpo de main a una clase Java (generarJavaDesdeArchivo): declaraciones, asignaciones, print y println, operadores e if. No traduce objetos, métodos, for, new ni interpolaciones.
Interpreter.rsc ejecuta el AST directamente e imprime en consola (ejecutar). Soporta aritmética, if, val y var, asignación, for con <- e in, métodos globales y recursivos, objetos con herencia, campos y métodos en el cuerpo, new con override (que ve las variables de main), interpolación y getClass.getSimpleName. print, println, List y range son funciones predefinidas. Ejemplo: ejecutar(|project://entrega2-ple/test/prueba.ol|); imprime 20. main (Main.rsc) verifica los tipos antes de ejecutar y se detiene si hay errores.

TypePal

Se agregó la dependencia en pom.xml y se subió Rascal a 0.43.1, porque la extensión de Rascal para VS Code exige la 0.43.0 o una superior.

```
<dependency> <!-- en <dependencies> de pom.xml -->
  <artifactId>rascal</artifactId>  <version>0.43.1</version>
</dependency>
<dependency>
  <artifactId>typepal</artifactId>  <version>0.17.2</version>
```

</dependency>

(groupId de ambas: org.rascalmpl)

Checker.rsc hace extend analysis::typepal::TypePal. TypePal resuelve variables y ámbitos y ordena las dependencias entre expresiones. Los objetos, la herencia y la varianza se guardan en tablas propias, porque el subtipado necesita recorrer la jerarquía de extends.

Verificación de tipos

Construcción	Regla
Literales	Int, Float, Boolean, Char, String; las cadenas verifican sus interpolaciones ${...} y $x.
val/var con anotación	El valor debe ser subtipo del tipo anotado.
val/var sin anotación	Se infiere el tipo del valor.
+ - * / **	Números; mezclar Int y Float da Float. + también concatena dos String. % solo acepta Int.
< > <= >=	Dos números o dos Char; resultado Boolean.
=, <>	Los operandos deben tener tipos comparables; resultado Boolean.
and, or, neg	Operandos Boolean.
if c then a else b	c es Boolean; el tipo es el ancestro común de a y b.
Asignación x = e	x debe existir y e debe ser subtipo del tipo de x.
Métodos	La última expresión del cuerpo debe ser subtipo del tipo de retorno declarado.
Llamadas	Mismo número de argumentos y cada uno subtipo del parámetro.

Construcción	Regla
new T(args) { override ... }	Argumentos contra el constructor; si T es genérico y no se dan tipos, se infieren (new Lista(new Perro) es Lista[Perro]); cada override coincide con la firma del método.
e.campo, e.metodo(args)	Se busca el miembro en el objeto y sus ancestros, sustituyendo los parámetros de tipo.
Subtipado	D extends B hace a D subtipo de B. Con C[+A], C[X] es subtipo de C[Y] si X es subtipo de Y; con C[-A] va al revés; con C[A] los tipos deben ser equivalentes.

Decisiones de diseño: no hay conversión implícita de Int a Float. print(x) toma el tipo de x, para que un método pueda terminar en print (como en snippet2). Una expresión ya reportada como error pasa a un tipo especial compatible con todo, para no encadenar errores.

Regla de existencia

Todo lo que una definición de objeto, un método o una expresión nombra debe existir. Los mensajes llevan el prefijo [Existencia]:

Los tipos de campos, parámetros, retornos y anotaciones son básicos, objetos definidos o parámetros de tipo declarados.

El objeto de extends existe, es un objeto y la herencia no es circular.

Obj.m = method ... solo se acepta si Obj existe.

new T exige que T exista; en override m, el método m debe existir en el objeto.

e.campo y e.metodo() exigen que el miembro exista en el tipo de e; las funciones llamadas deben estar definidas.

Las variables no declaradas las reporta TypePal como Undefined variable.

También se rechazan definiciones duplicadas y objetos genéricos usados sin argumentos de tipo o con un número incorrecto.

Pruebas

Desde la terminal de Rascal en VS Code: import Checker; y luego validar(|project://entrega2-ple/test/ARCHIVO|);. Para ejecutar: import Interpreter; y luego ejecutar(|project://entrega2-ple/test/ARCHIVO|);. Para verificar y ejecutar en un solo paso: import Main; y luego main(archivo=|project://entrega2-ple/test/ARCHIVO|);.

Archivo	Resultado verificado
test/tipos_ok.ol	OK (tipos básicos, operadores, if, interpolación)
test/tipos_error.ol	8 errores (líneas 2 a 8 y 10)
test/objetos_ok.ol	OK (herencia, genéricos, varianza, métodos, override, new)
test/objetos_error.ol	15 errores (líneas 3, 5, 6, 11 a 21 y 23); 8 son [Existencia]
test/varianza_error.ol	2 errores (líneas 11 y 15): viola covarianza y contravarianza
test/asig_ok.ol	OK (asignaciones bien tipadas; val b = x = 2 sigue siendo igualdad)
test/asig_error.ol	3 errores (líneas 3, 4 y 6): tipo incompatible y variable no definida
test/snippet3_ok.ol	OK: snippet3 con Transformador.transformar corregido
instance/snippet1.obl	OK: Lista[Perro] es subtipo de Lista[Animal]
instance/snippet2.obl	OK: define Gato, asigna Gato.hablar y tiene main
instance/snippet3.obl	3 errores [Existencia]: el enunciado tiene un error de nombre: define Transformar.transformar, pero el objeto se llama Transformador
ejecutar test/prueba.ol	Imprime 20
ejecutar test/objetos_ok.ol	Imprime Rex / Labrador / Perro (campos, herencia, método y override)

Archivo	Resultado verificado
ejecutar test/snippet3_ok.ol	Imprime el mensaje del override y Tipo de comida resultante: ComidaPerro
main test/tipos_error.ol	Muestra los 8 errores de tipos y no ejecuta el programa
main test/if_precedencia.ol	Imprime 1 y 12: el else de un if abarca todo lo que tiene a la derecha (else 2 + 10)

Limitaciones

Los campos declarados en el cuerpo de un objeto deben llevar anotación de tipo.

Un método no ve como variables los campos que hereda de su padre (sí puede accederlos con obj.campo).

El lenguaje no define colecciones: las variables de un for no tienen tipo y se aceptan en cualquier uso.

La asignación solo acepta una variable a la izquierda (no obj.campo = e), y no se verifica que la variable sea var y no val.

El ; entre sentencias es obligatorio (sección 3.1) y la última sentencia de un cuerpo no lleva ;.

Los operadores -> y => no tienen semántica definida en el enunciado; se verifican sus operandos y no se restringe su resultado.

snippet4 y colores.obl son fragmentos con nombres no definidos (a, b, pares...), así que no pasan el verificador completo; sirven para probar el parser y el resaltado.

JavaGenerator solo traduce el cuerpo de main; objetos, métodos, for, new e interpolaciones no se traducen.

Interpreter.ejecutar no verifica tipos por sí solo; main (Main.rsc) llama primero a verificar. range y List son funciones predefinidas y el verificador no restringe su resultado, porque el lenguaje no define tipos de colección.

Origen de la solución

La verificación de tipos (Checker.rsc) y las pruebas son desarrollo propio del equipo; no se tomó de otro grupo. La gramática con tipos y el parser se desarrollaron en el repositorio del equipo.